```
ok = require('assert').ok;

if (true) {
  (function(x, y) {
    console.log('let expression');
    return console.log('wraps block in closure');
  })(2, 2);
}

ok(typeof x === "undefined" || x === null);

ok(typeof y === "undefined" || y === null);
```

Scruffy achieved the function closure by rewriting the token stream so that the `let` expression is replaced with a `do` that has the `let` variables as parameters. Any code at the same level of indentation as the `let` is then scoped inside the `let`:

```
do (x = 2, y = 2) ->
  console.log 'let expression'
  console.log 'wraps block in closure'
```

Scruffy's implementation of `let` is limited. It supports only one `let` per block and it must be the first line. Also, because `let` is a word reserved by the CoffeeScript compiler, Scruffy had to rewrite the raw source before passing it to the tokenizer.

Listing 8.9. Scruffy's `let` implementation using a custom rewriter